

PIV Project

Group 9	
Alexandre Cachera	116285
Tiago Nunes Couto	113311
Bernardo Silva Costa	102777
Gabriel Badan	116537

1 Motivations

This project aims to estimate geometric transformations between a reference image and a sequence of images using classical and modern computer vision techniques. The project is structured in two parts: (1) *Part 1*: 2D homography estimation from grayscale images, and (2) *Part 2*: rigid 3D transformation (rotation and translation) from RGB-D data. These techniques find applications in object tracking, 3D reconstruction, robotic localization, and augmented reality.

2 Part 1: Planar Object Tracking via Homography

This section describes the implementation of a 2D planar tracking system that estimates the Homography (H) between a reference template and a sequence of video frames.

2.1 Algorithmic Details

2.1.1 Feature Extraction and Matching

The system utilizes the **Scale-Invariant Feature Transform (SIFT)** [1] to detect interest points robust to scale

and rotation, and extract 128-dimensional invariant descriptors. We selected SIFT over simpler alternatives (Harris corners, ORB) because the document occupies varying image regions as the camera moves, requiring scale-invariant detection. Additionally, SIFT handles arbitrary document orientations through its rotation-invariant formulation, and unlike learning-based methods, it requires no training data and remains fully interpretable. *SIFT Parameters*: OpenCV defaults ($\sigma = 1.6$, standard octaves and scales).

Then, feature matching is performed between the template and each frame using the following strategy:

We use Euclidean distance metric combined with **Lowe's ratio test** to filter ambiguous matches. For each descriptor in the frame, we find the 2 nearest neighbors in the template. If $\text{distance}(\text{NN}_1) < 0.75 \times \text{distance}(\text{NN}_2)$, we accept the match.

The selection of the 0.75 threshold for the distance ratio test is grounded in D. Lowe's original empirical analysis [1]. This test aims to eliminate ambiguous matches by comparing the Euclidean distance of the nearest neighbor to that of the second-nearest neighbor. A ratio exceeding 0.75 indicates that the two best candidates are similarly close, suggesting that the feature lacks distinctiveness—often due to repetitive patterns or background clutter.

We chose this specific value because it offers an optimal trade-off between precision and recall. **Higher thresholds** (e.g., 0.9) tend to retain too many incorrect matches where the descriptor is not unique enough, while **stricter thresholds** (e.g., 0.5) are too aggressive and discard valid correspondences, particularly in regions with lower texture variations. Thus, 0.75 effectively filters out unreliable keypoints while maintaining a dense set of features for the subsequent RANSAC estimation.

2.1.2 Homography Estimation (DLT with Normalization)

The relationship between planar points in the template ($\mathbf{x}$) and the frame ($\mathbf{x}'$) is modeled by a 3×3 Homography matrix H such that $\mathbf{x}' \sim H\mathbf{x}$. The **Direct Linear Transform (DLT)** algorithm is used to solve for H using Singular Value Decomposition (SVD).

Data Normalization A crucial step implemented is isotropic normalization [2]. Before applying DLT, points are translated so their centroid is at the origin and scaled so the average distance from the origin is $\sqrt{2}$. This improves the numerical stability of the SVD.

$$T_{norm} = \begin{bmatrix} s & 0 & -s \cdot c_x \\ 0 & s & -s \cdot c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1)$$

Matrix A Construction For N correspondence points $(x_i, y_i) \rightarrow (u_i, v_i)$, create a $2N \times 9$ matrix:

$$A_i = \begin{pmatrix} -x_i & -y_i & -1 & 0 & 0 & 0 & x_i u_i & y_i u_i & u_i \\ 0 & 0 & 0 & -x_i & -y_i & -1 & x_i v_i & y_i v_i & v_i \end{pmatrix}$$

SVD and H Extraction The solution is found via SVD: $A = U\Sigma V^T$, where H is reshaped from the last column of V (corresponding to the smallest singular value).

Denormalization The final Homography is denormalized via:

$$H = T_{dst}^{-1} H_{norm} T_{src} \quad (2)$$

2.1.3 Robust Estimation (RANSAC)

Even after Lowe's ratio test, some incorrect correspondences persist—particularly in low-texture regions or near occlusion boundaries. A direct least-squares solution to the homography would be severely biased by these outliers, potentially producing unusable results. RANSAC addresses this by iteratively searching for the largest subset of geometrically consistent matches.

Our implementation uses the following parameters: **Iterations** (5000), **Sample Size** (4 points, the algebraic minimum for homography), **Inlier Threshold** (3.0 pixels), and **Refinement** (after identifying the best consensus set, H is re-computed using all inliers for improved accuracy).

Inlier Threshold Selection The reprojection error threshold was set to 3.0 pixels based on an empirical analysis of the localization error inherent in our video data. At the typical working distances and camera resolutions used in this project, a deviation of 3.0 pixels corresponds to approximately 1–2 mm on the physical document surface. This specific value allows the system to accommodate the sensor noise floor and minor feature localization inaccuracies, while effectively rejecting correspondence errors caused by occlusions or texture ambiguities. A lower threshold would discard valid matches due to discretization noise, whereas a higher threshold would admit outliers, degrading the stability of the estimated homography.

Iteration Strategy To ensure robust model estimation without compromising runtime performance, we implemented an iteration scheme with 5000 iterations. The theoretical number of iterations needed to achieve a confidence probability p is:

$$k = \frac{\log(1-p)}{\log(1-w^s)} \quad (3)$$

where w is the inlier ratio and $s = 4$ is the sample size. With $p = 0.99$, this ensures a 99% chance of sampling a minimal set of outlier-free points. The 5000-iteration budget provides a computational margin for difficult frames where the inlier ratio may be low (e.g., during severe motion blur or occlusion).

2.2 Experimental Setup and Evaluation

Dataset We used the TAAG dataset, which provides a reference template image and a sequence of frames with varying perspectives and occlusions.

Evaluation metrics We considered the *number of inliers* as an indicator of matching quality and *robustness* as an ability to handle partial occlusions and significant transformations.

Execution Pipeline The implementation is organized in two stages: (1) **Template Initialization:** SIFT extracts features from the template image and saves them to `template_features.mat`. (2) **Frame-by-Frame Processing:** For each frame, SIFT features are extracted and stored; feature matching via Lowe’s ratio test identifies correspondences; RANSAC (5000 iterations, 3.0 pixel threshold) estimates the best homography; the final H is refined using all inliers and saved as `homography_NNNN.mat`. Frames with fewer than 4 matches are skipped.

2.3 Example Results

We evaluated the algorithm on a 14-frame sequence with varying conditions.

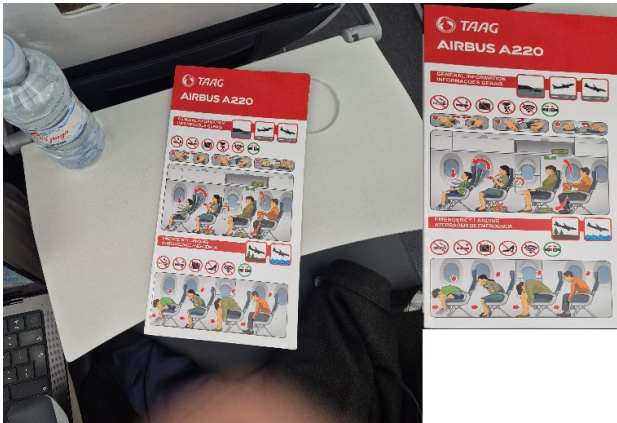


Figure 1: Successful tracking under perspective change.

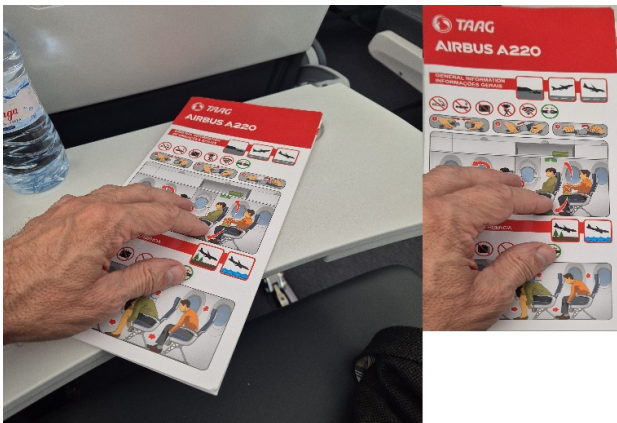


Figure 2: Robust tracking despite partial occlusion—RANSAC rejects outliers.



Figure 3: Failure case: insufficient correspondences produce incorrect homography.

Quantitative Analysis Across the 14 processed frames:

- **Raw Matches (after ratio test):** 50–100 correspondences per frame
- **RANSAC Inliers:** 40–80 (80–85% acceptance rate)
- **Mean Reprojection Error (inliers):** < 1.5 pixels

Key Observations

- The algorithm successfully tracks the template under significant perspective changes (up to 40° rotation).
- Partial occlusions are handled effectively: RANSAC identifies the occluding object’s matches as outliers and excludes them from the homography computation.
- Failure occurs when the document is mostly out of frame or when lighting conditions drastically reduce feature detectability. In such cases, fewer than 4 valid correspondences remain, and the resulting homography is unreliable.

3 Part 2: 3D Rigid Body Motion Estimation

This section details the estimation of the camera’s 3D trajectory (Odometry) by calculating the Rigid Body Transformation (R, T) between the reference template and each frame.

3.1 Algorithmic Details

3.1.1 Feature Extraction and Matching

Feature matching uses SIFT on RGB images to identify 2D-2D correspondences, applies Lowe’s ratio test (threshold = 0.75), and recovers descriptors robust to lighting changes.

3.1.2 3D Point Generation

Unlike Part 1, this module utilizes RGB-D data. 2D key-points (u, v) are back-projected into 3D space (X, Y, Z) using the corresponding depth map and the Pinhole Camera Model intrinsics matrix K :

$$Z = \text{depth}(v, u) \quad (4)$$

$$X = \frac{(u - c_x) \cdot Z}{f_x}, \quad Y = \frac{(v - c_y) \cdot Z}{f_y} \quad (5)$$

Points with invalid depth ($Z \leq 0$ or NaN) are discarded.

3.1.3 Orthogonal Procrustes Analysis

To find the rotation R and translation T that align two clouds of 3D points ($P_B \approx R \cdot P_A + T$), we use the Orthogonal Procrustes method [3]. Unlike iterative approaches such as ICP (Iterative Closest Point), Procrustes assumes that point correspondences are already established (via SIFT matching). This allows a direct closed-form solution rather than an iterative search, resulting in faster computation and guaranteed convergence. The method proceeds as follows:

1. Compute centroids: $\bar{P}_A = \frac{1}{N} \sum P_A^{(i)}, \bar{P}_B = \frac{1}{N} \sum P_B^{(i)}$
2. Center both point clouds: $P'_A = P_A - \bar{P}_A, P'_B = P_B - \bar{P}_B$
3. Compute cross-covariance matrix: $H = (P'_A)^T \cdot P'_B$
4. Decompose via SVD: $H = U \Sigma V^T$
5. Compute rotation: $R = VU^T$
6. If $\det(R) < 0$, negate the last column of V and recompute R [4]
7. Compute translation: $T = \bar{P}_B - R \cdot \bar{P}_A$

3.1.4 RANSAC for 3D Alignment

Depth sensors introduce noise near object boundaries, and SIFT matches can fail in textureless or specular regions. Applying Procrustes directly to all correspondences would be corrupted by these outliers. We therefore wrap the Procrustes solver in a RANSAC loop with: **Iterations** (2000), **Sample Size** (4 points, providing redundancy over the theoretical minimum of 3), and **Threshold** (0.05 meters = 5 cm), applied to the Euclidean distance between transformed source and target points.

Threshold Selection The 5 cm threshold was determined through empirical analysis of our RGB-D sensor. Correctly matched points typically exhibit errors within this range due to sensor quantization, while mismatches or boundary artifacts produce errors exceeding 10 cm. After finding the largest consensus set, the final (R, T) is recomputed using all inliers.

3.2 Experimental Setup and Evaluation

Dataset The dataset comprises 1 RGB template with depth (templatedepth.mat), multiple RGB-D frames (frame_NNNN.mat), typical resolution of 640×480 pixels, and depth format as $H \times W$ matrix with values in millimeters.

Evaluation metrics Metrics include: **Number of Valid 3D Correspondences** (indicator of matching quality and depth density), **RANSAC Inliers** (ratio of robustness in pose estimation), and **Average 3D Error** (for inlier points).

Execution Pipeline The system iterates through pairs of RGB and Depth files: (1) Load RGB template with depth; (2) For each frame—extract and match SIFT features, back-project matched 2D points to 3D using depth maps, run RANSAC + Procrustes to estimate R, T , and store results in transform_NNNN.mat files.

3.3 Example Results

The resulting R and T matrices describe the camera’s motion relative to the template. The use of RANSAC signif-

icantly reduces “jumps” in the trajectory caused by noisy depth values at object boundaries.

Typical result for a frame:

- **2D-2D Correspondences:** ~ 60 – 150 SIFT matches
- **Valid 3D Correspondences** (depth available): ~ 50 – 120
- **RANSAC-Procrustes Inliers:** ~ 40 – 100 (70–85% acceptance)
- **Average Error (inliers):** 1–3 cm

Output File: `transform_NNNN.mat` contains R (3×3 orthogonal rotation matrix) and T (3×1 translation vector).

Validation:

- Matrix R satisfies $R^T R = I$ (orthogonality)
- $\det(R) = 1$ (pure rotation, no reflection)
- Vector T represents translation in the depth sensor’s unit system

4 Discussion and Limitations

4.1 Comparison of Methods

2D Homography (Part 1) is efficient for tracking planar objects but fails on non-planar targets or when camera motion induces significant parallax that a planar model cannot represent. DLT normalization is critical for maintaining accuracy with large coordinates. **3D Procrustes (Part 2)** provides full 6-DoF pose estimation and requires depth data, making it dependent on depth sensor quality. The implementation correctly handles coordinate system reflections, a common pitfall in SVD-based estimation.

4.2 Identified Limitations

Part 1 Limitations include: limited to planar scenes, sensitivity to lighting changes (though SIFT mitigates this), and requirement for sufficient extractable features.

Part 2 Limitations include: critical dependency on depth data quality (noise, holes), degraded performance if valid 3D correspondences < 4 (RANSAC sample size), and sensitivity to calibration of K .

4.3 Conclusion

The implemented pipeline demonstrates a standard computer vision approach: Feature Extraction $\rightarrow$ Matching $\rightarrow$ Robust Model Estimation. The use of SVD for both DLT and Procrustes ensures mathematically rigorous solutions to the minimization problems.

References

- [1] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.
- [2] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2003.
- [3] K. S. Arun, T. S. Huang, and S. D. Blostein, “Least-squares fitting of two 3-D point sets,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. PAMI-9, no. 5, pp. 698–700, 1987.
- [4] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, 1991.